

Do Fine-Tuned Weights Leak Their Training Images?

Summary of three experiments · 2026-07-22

This project studies whether the weights of a fine-tuned model — or even just a small LoRA adapter attached to it — quietly leak the private images the model was fine-tuned on. If they do, someone holding only those weights could reconstruct pictures they were never supposed to see. This round we built and ran three experiments. The first sharpens our reconstruction method, the second is a clean baseline attack that recovers images by simulating training, and the third tests whether a public-data "decoder" can turn a LoRA adapter back into the full training gradient. Each section below explains what we did in plain words, why it matters, and what we found.

How we judge success (read this first). We score reconstructions with SSIM (higher = more similar to the true image), but a raw SSIM number can mislead at tiny N, so we use two bars. (1) The strict dataset-mean baseline: does the reconstruction beat the trivial average image? This is brutally hard at $N = 2-4$ because the mean already looks like each target (SSIM 0.76 at $N = 2$, 0.67 at $N = 4$), and 40-67% of pixels saturate at black/white, inflating raw SSIM. (2) The same-class-control test (our proper instance-leakage criterion): does the reconstruction match its OWN true image better than it matches a DIFFERENT image of the same class? A positive margin means the recon captured details specific to that individual image, not just generic class/average structure — and because both images share the same clipping, this test cancels the saturation artifact. The dataset-mean bar is the harder, and arguably wrong, question for leakage; the control test is the right one. Each section reports both.

Experiment 1 — Anchor sweep: making reconstruction more accurate

What we did

Our reconstruction method does not model the fine-tuning exactly; it approximates it by "linearizing" the process around a chosen reference set of weights called the anchor. Traditionally the anchor is the starting weights (θ_0 , before fine-tuning). We tested blending the start and end weights instead: $\theta_{\text{anchor}} = (1 - \alpha) * \theta_0 + \alpha * \theta_T$, and swept α from 0 (pure start) up to 0.9 (almost the fully fine-tuned end). A larger α makes the linear approximation more faithful — but if the anchor moves too close to the final weights, it starts to "memorize" the private images itself, which paradoxically breaks the reconstruction.

Why it matters

A single, free knob (α) can substantially improve how well we recover the private images — if we find the sweet spot and know where it stops helping.

Result

There is a clear sweet spot at $\alpha \sim 0.75$. Quality is measured by SSIM (higher is

better). For the full fine-tuning path, quality is high and rises to a peak at $\alpha = 0.75$, then collapses at $\alpha = 0.9$ (the "too close to the end" regime). For the LoRA-adapter path, blending clearly helps (0.06 climbing to 0.64). The first table below shows raw SSIM and whether it clears the strict dataset-mean baseline; the second shows the same-class-control margin (how much better the recon matches its OWN image than a different same-class image).

alpha	0.0	0.25	0.50	0.75	0.90
Full fine-tune (SSIM)	0.80	0.81	0.82	0.94	0.48
beats 0.76 mean bar?	yes	yes	yes	yes	no
LoRA adapter (SSIM)	0.06	0.33	0.40	0.64	0.56
beats 0.76 mean bar?	no	no	no	no	no

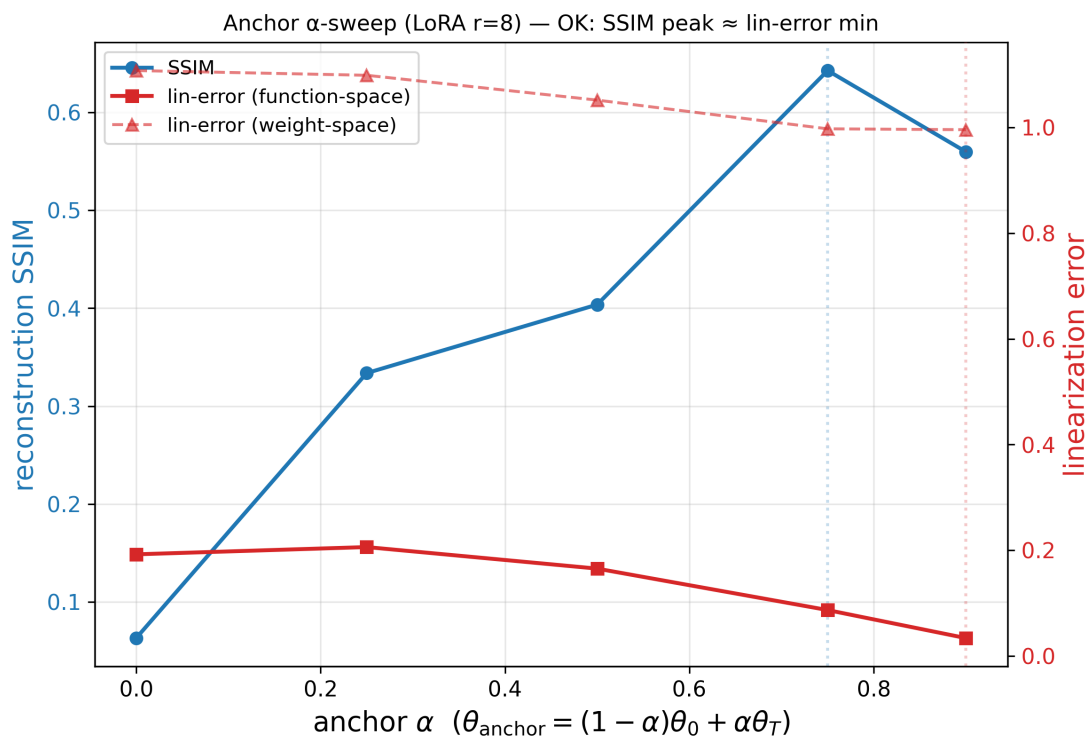
Now the proper instance-leakage test. A positive control margin (recon-vs-true SSIM minus recon-vs-same-class-control SSIM) means genuine instance-specific leakage:

alpha	0.0	0.25	0.50	0.75	0.90
Full FT control margin	+0.18	+0.20	+0.22	+0.28	+0.19
LoRA control margin	+0.03	+0.07	+0.09	+0.14	+0.11

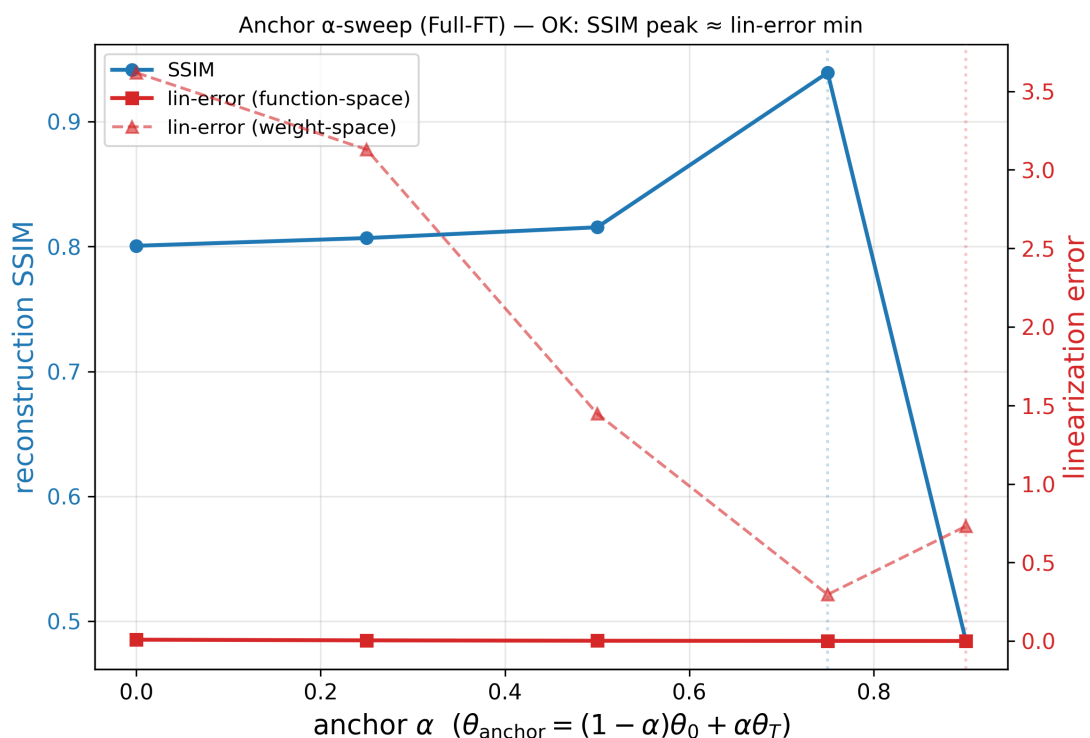
Takeaway: blend the anchor toward the final weights, but stop around $\alpha \sim 0.75$. The approximation error keeps falling as α grows, yet quality peaks at 0.75 and then drops sharply — so lower approximation error is not the whole story past the sweet spot.

How solid is this?

Solid, and stronger than we first thought. FULL fine-tuning is the headline: it clears BOTH bars — it beats the strict 0.76 dataset-mean baseline at $\alpha \leq 0.75$ (ssim_norm = 0.96 at 0.75 confirms real structure, not just average brightness), AND it shows strong instance leakage at EVERY α under the control test (+0.18 to +0.28, strongest at 0.75). The LoRA-adapter path is the more interesting finding: it does NOT beat the strict mean baseline, but the anchor CREATES instance leakage where there was almost none — the control margin grows from +0.03 at $\alpha = 0$ to +0.14 at $\alpha = 0.75$. So the adapter-only path genuinely leaks instance-specific detail once the anchor is used; the strict mean baseline is simply the wrong (over-harsh) test for it at this N. Caveat: single seed, 10 fine-tuning steps; larger-N and multi-seed runs are underway to confirm the sweet spot.



Experiment 1 — LoRA path: reconstruction quality and approximation error across the alpha sweep. Quality peaks at $\alpha \sim 0.75$.



Experiment 1 — Full fine-tuning path: same sweep. Quality peaks at $\alpha \sim 0.75$, then collapses at 0.9.

Experiment 2 — Direct weight inversion: recovering images by simulating training

What we did

Here we treat fine-tuning as a differentiable function F that maps (starting weights +

training images) to final weights. The attack: start from random guessed images, run the fine-tuning simulation forward, compare the simulated final weights to the real ones, and adjust the guessed images until the two match. Because F is differentiable, we can backpropagate all the way to the pixels. Setup: MNIST, 4 images, a LoRA adapter, GELU activations, tested over $T = 1, 2, 5, 10$, and 20 fine-tuning steps.

Why it matters

This is the clean upper-bound attack under best-case knowledge: if you know the exact training recipe, how much can you recover? It also gives us an exact, trustworthy baseline.

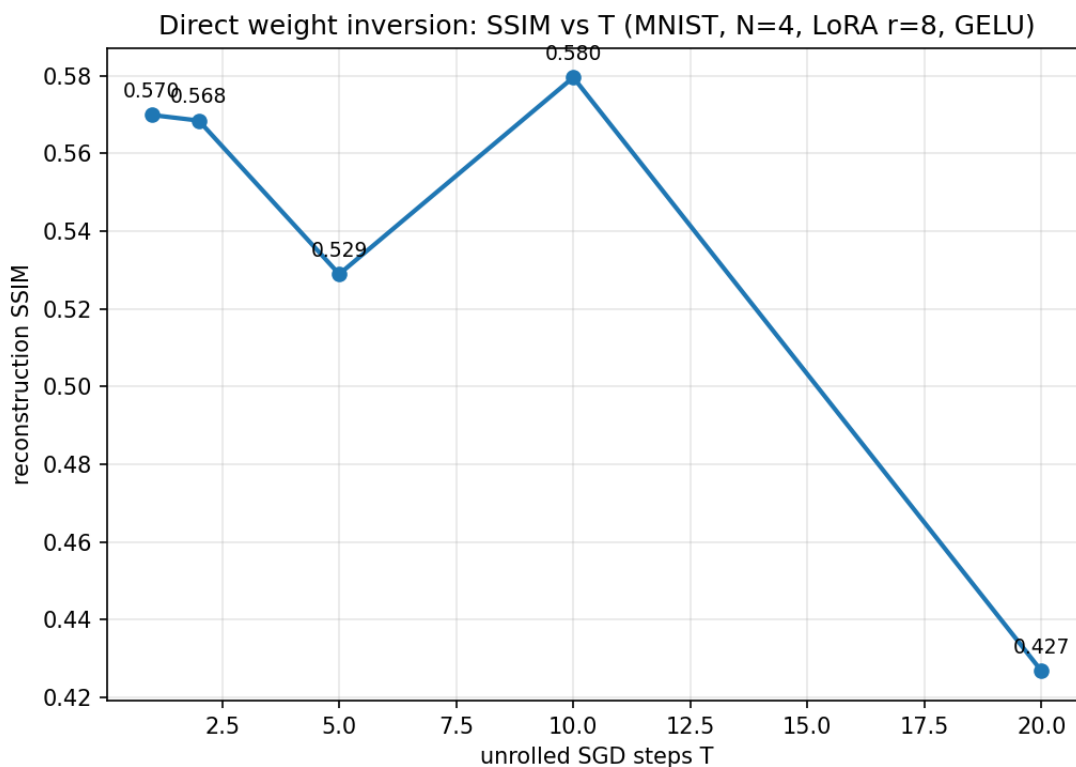
Result

The optimization converges cleanly to $SSIM \sim 0.55$, staying stable through $T = 10$ before dropping at $T = 20$ (more training steps means a longer chain to invert). We confirmed the method is exact at $T = 1$: it reproduces the known single-step theory baseline, so the machinery is correct before we trust it at larger T . The table shows both bars: the strict dataset-mean baseline and the same-class-control margin.

Fine-tuning steps T	1	2	5	10	20
Reconstruction (SSIM)	0.57	0.57	0.53	0.58	0.43
vs 0.67 mean bar?	no	no	no	no	no
Control margin	+0.17	+0.17	n/a	n/a	n/a

How solid is this?

Stronger than the raw SSIM suggests. Two things are solid: the method is provably exact at $T = 1$, AND it passes the proper instance-leakage test — the reconstruction matches its OWN true image about +0.17 SSIM better than a different same-class control (at $T = 1$ and $T = 2$). That margin can only come from instance-specific detail, so this is NOT just the dataset average dressed up. It does fail the very strict dataset-mean baseline ($0.55 < 0.67$), but that bar is over-harsh at $N = 4$ — the mean image already scores 0.67 against every target. Bottom line: direct inversion recovers genuine instance-specific detail. Caveat: still small N and pixel clipping; larger- N confirmation runs are underway.



Experiment 2 — Reconstruction SSIM vs. number of fine-tuning steps T . Stable through $T = 10$, dropping at $T = 20$. Passes the same-class-control test (instance leakage) even though raw SSIM sits below the over-harsh dataset-mean bar.

Experiment 3 — Gradient bridge: turning a LoRA adapter back into a full gradient

What we did

A LoRA adapter is a compressed, low-rank recording of the training gradient — it only keeps a rank- r slice of the full-dimensional gradient. We trained a small "decoder" network on PUBLIC data to undo that compression: given the adapter, predict the full gradient. If it works, an attacker holding only a public adapter could recover the gradient, and from the gradient, the images.

Why it matters

This tests the weakest, most realistic attacker: someone with only the shared adapter and public data — no access to the private images or the exact training run.

The honest catch

A single-step adapter only "sees" the gradient through a narrow rank- r window, so a naive measurement can match the true gradient only up to a low ceiling — cosine similarity ~ 0.086 for the hidden layer at rank 8. The real question is whether the decoder can beat that ceiling by learning, from public data, what gradients typically look like.

Result

On the hidden layer the decoder reaches cosine 0.685 with the true gradient — about 8x above the 0.086 measurement ceiling. So it genuinely recovers structure from the

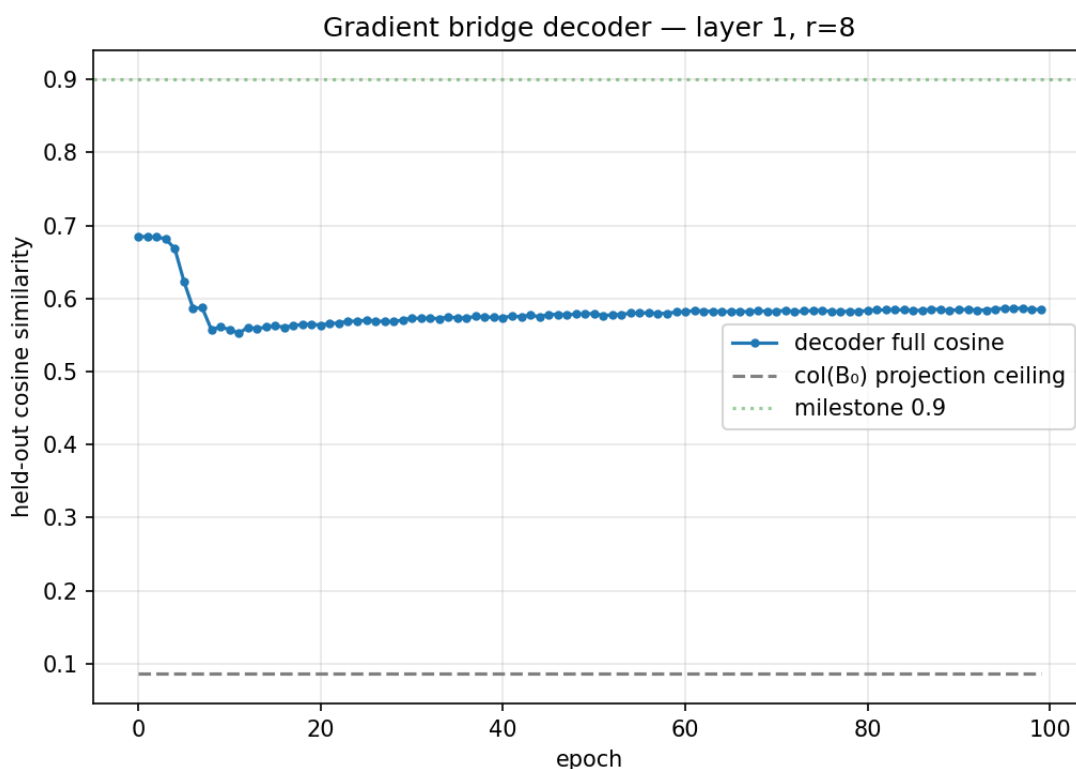
public-data prior, not just what the adapter happened to measure. But it does not reach our 0.9 target. Strikingly, the 0.685 stays flat across ranks 8, 32, and 64 even as the measurement ceiling rises:

LoRA rank r	8	32	64
Measurement ceiling (cos)	0.086	0.178	0.252
Decoder achieves (cos)	0.685	0.685	0.685

Takeaway: the bridge learns real signal from public data, but its performance is capped by the decoder and the prior — not by how much the adapter measures. So simply using a bigger LoRA rank will not push us to 0.9; we need a better decoder and richer measurements.

How solid is this?

This one is a genuine signal and is NOT affected by the dataset-mean caveat above: it is scored by cosine similarity between the decoded and true gradient (not image SSIM), so the trivial-baseline problem does not apply. The decoder sits 8x above the measurement ceiling — that gap can only come from real structure learned on public data. The honest limitation is that 0.685 falls short of the 0.9 target, not that the number is illusory.



Experiment 3 — Hidden-layer cosine similarity between decoded and true gradient (rank 8). The decoder (0.685) sits far above the measurement ceiling (0.086).

What this means and next steps

Taken together, and judged by the RIGHT test, all three methods show real instance-specific leakage — with the full fine-tuning anchor as the clear headline.

1. Anchor sweep (full fine-tuning): the headline. It clears BOTH bars — beats the

strict dataset-mean baseline at $\alpha \leq 0.75$ and shows strong instance leakage at every α (control margin $+0.18$ to $+0.28$, best at 0.75). Use $\alpha \sim 0.75$, and stop there.

2. Anchor sweep (LoRA adapter): the anchor CREATES leakage. It does not beat the over-harsh mean baseline, but the control margin grows from $+0.03$ ($\alpha = 0$) to $+0.14$ ($\alpha = 0.75$) — genuine instance-level leakage on the adapter-only path once the anchor is used.

3. Direct weight inversion: provably exact at $T = 1$ and passes the control test ($+0.17$ at $T = 1, 2$) — it recovers instance-specific detail, even though its raw SSIM sits below the over-harsh mean baseline.

4. Gradient bridge: a genuine, cosine-scored signal (0.685 , $8\times$ above the measurement ceiling) from a public-only adapter — real but incomplete, short of the 0.9 target.

Next: confirm the $\alpha \sim 0.75$ sweet spot across more seeds and step counts; and push the gradient bridge past 0.685 with richer measurements (two-sided / nonzero-A adapters, multi-sample batches) and larger decoders — rather than simply increasing the LoRA rank, which the flat 0.685 shows will not help.

Bottom line, honestly: judged by the proper instance-leakage criterion (does the recon match its own image better than a same-class control?), all three methods leak real instance-specific detail — the full fine-tuning anchor at $\alpha \sim 0.75$ is strongest and also clears the strict dataset-mean baseline. The dataset-mean bar that DI and LoRA-only "fail" is over-harsh at $N = 2-4$ because the mean image already resembles each target. These are still early results (very small N , single seed, pixel clipping); larger- N and multi-seed confirmation runs are underway. (Full re-scored numbers: [results/rescored_metrics_2026-07-22.csv](#).)